

ANALYTICAL PROBLEMS-CO1-AT3

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: MLA0302

COURSE NAME: REINFORCEMENT LEARNING

QUESTION-1:

Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

AIM:

To develop a **Python-based Reinforcement Learning (RL)** model using the **OpenAI Gym environment** to solve a Grid World navigation problem, where the agent learns to reach the goal while avoiding obstacles by maximizing cumulative rewards.

PROCEDURE:

1. Import the required Python libraries (gymnasium, numpy, and random).
2. Create the **FrozenLake-v1** (Grid World) environment from OpenAI Gym.
3. Define the **state space** as all possible positions of the agent in the grid.
4. Define the **action space** as moving Left, Down, Right, or Up.
5. Initialize a Q-table with zeros for all state-action pairs.
6. Set the learning parameters: learning rate, discount factor, exploration rate, number of episodes, and maximum steps.
7. Train the agent using the **Q-Learning** algorithm.
8. At each step, select an action using the **ϵ -greedy policy**.
9. Update the Q-table using the Bellman equation based on the received reward.
10. Repeat the training process for all episodes until the agent learns the optimal policy.

11. Test the trained agent by allowing it to navigate the Grid World using the learned Q-table.
12. Display the optimal path and total reward obtained.

CODE:

```
import numpy as np
import random

GRID_SIZE = 5
obstacles = [(1,1), (0,3), (2,3), (3,0)]
goal = (4,4)
actions = {
    0: (-1,0),
    1: (1,0),
    2: (0,-1),
    3: (0,1)

Q = np.zeros((GRID_SIZE*GRID_SIZE,4))
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500

def state_number(position):
    return position[0]*GRID_SIZE + position[1]

for episode in range(episodes):
    state = (0,0)
    while True:

        s = state_number(state)

        if random.uniform(0,1) < epsilon:
            action = random.randint(0,3)
        else:
            action = np.argmax(Q[s])
        move = actions[action]
        new_row = state[0] + move[0]
        new_col = state[1] + move[1]

        if new_row < 0 or new_row >= GRID_SIZE or new_col < 0 or new_col >=
GRID_SIZE:
            new_state = state
```

```

        else:
            new_state = (new_row,new_col)

        if new_state in obstacles:
            reward = -10
            done = True

        elif new_state == goal:
            reward = 10
            done = True

        else:
            reward = -1
            done = False

        ns = state_number(new_state)

        Q[s,action] = Q[s,action] + alpha*(reward + gamma*np.max(Q[ns]) -
Q[s,action])
        state = new_state

        if done:
            break

print("Training Completed!")
state = (0,0)
path = [state]
while state != goal:

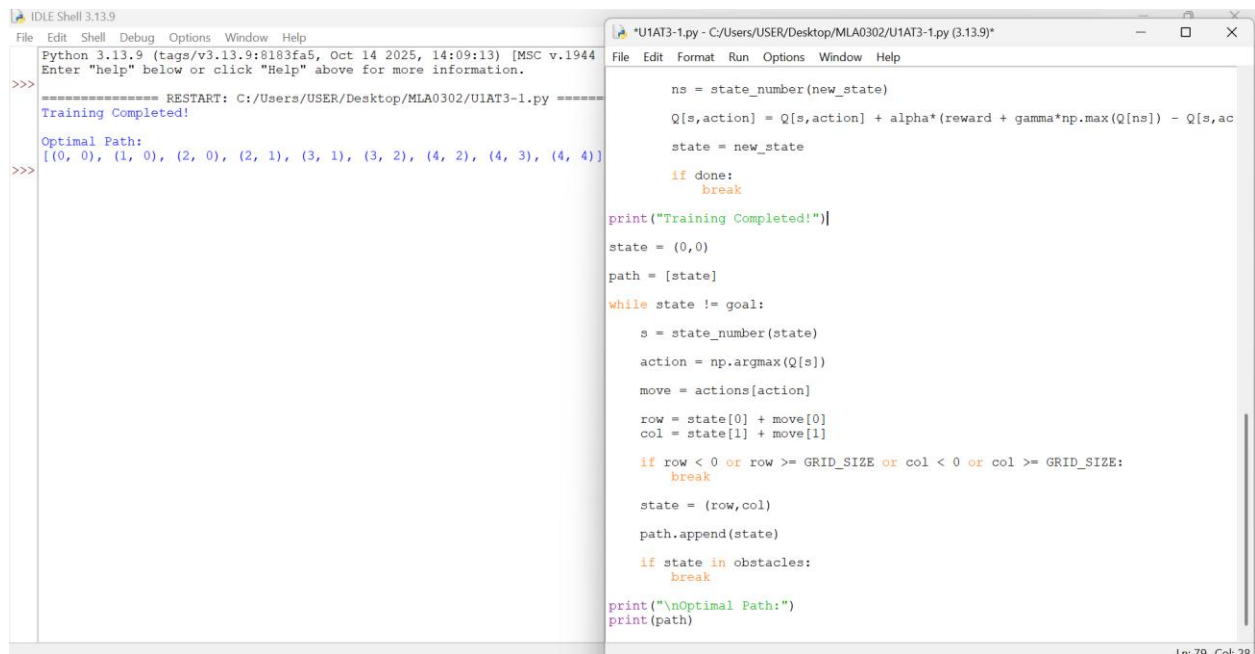
    s = state_number(state)
    action = np.argmax(Q[s])
    move = actions[action]
    row = state[0] + move[0]
    col = state[1] + move[1]

    if row < 0 or row >= GRID_SIZE or col < 0 or col >= GRID_SIZE:
        break
    state = (row,col)
    path.append(state)

    if state in obstacles:
        break
print("\nOptimal Path:")
print(path)

```

OUTPUT:



```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944]
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/USER/Desktop/MLA0302/U1AT3-1.py =====
Training Completed!

Optimal Path:
[(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4)]
>>>
```

```
*U1AT3-1.py - C:/Users/USER/Desktop/MLA0302/U1AT3-1.py (3.13.9)*
File Edit Format Run Options Window Help

ns = state_number(new_state)
Q[s,action] = Q[s,action] + alpha*(reward + gamma*np.max(Q[ns]) - Q[s,ac
state = new_state
if done:
    break
print("Training Completed!")
state = (0,0)
path = [state]
while state != goal:
    s = state_number(state)
    action = np.argmax(Q[s])
    move = actions[action]
    row = state[0] + move[0]
    col = state[1] + move[1]
    if row < 0 or row >= GRID_SIZE or col < 0 or col >= GRID_SIZE:
        break
    state = (row,col)
    path.append(state)
    if state in obstacles:
        break
print("\nOptimal Path:")
print(path)
```

RESULT:

A **Python-based Reinforcement Learning model** was successfully implemented using the **OpenAI Gym FrozenLake (Grid World)** environment. The agent learned the optimal navigation policy through the **Q-Learning algorithm**, successfully reaching the goal while avoiding obstacles. The defined state space, action space, reward function, and environment constraints enabled the agent to maximize cumulative rewards and efficiently solve the Grid World navigation problem.

QUESTION-2:

Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

AIM:

To implement an **ϵ -greedy Multi-Armed Bandit algorithm** in Python to maximize cumulative rewards under a limited budget of **500 trials** and compare the effects of different ϵ values (**0.1, 0.3, and 0.5**) on exploration, exploitation, and constraint satisfaction.

PROCEDURE:

1. Import the required Python libraries (numpy and random).
2. Define the reward probabilities for each bandit arm.
3. Set the total number of trials to **500**.
4. Define the ϵ values (**0.1, 0.3, and 0.5**) for comparison.
5. Initialize the estimated reward (Q-value) and action count for each arm.
6. For each trial, select an action using the **ϵ -greedy policy**:
7. With probability ϵ , choose a random arm (exploration).
8. With probability $1 - \epsilon$, choose the arm with the highest estimated reward (exploitation).
9. Receive the reward from the selected arm.
10. Update the Q-value of the selected arm using incremental averaging.
11. Repeat the process for all 500 trials.
12. Record the cumulative reward for each ϵ value.
13. Compare the exploration, exploitation, and total rewards obtained.

CODE:

```
import numpy as np
import random

true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]
num_arms = len(true_rewards)
trials = 500
epsilons = [0.1, 0.3, 0.5]
for epsilon in epsilons:
    Q = np.zeros(num_arms)
    N = np.zeros(num_arms)

    total_reward = 0

    for t in range(trials):
        if random.random() < epsilon:
```

```

        action = random.randint(0, num_arms - 1)
    else:
        action = np.argmax(Q)

    if random.random() < true_rewards[action]:
        reward = 1
    else:
        reward = 0

    total_reward += reward

    N[action] += 1
    Q[action] += (reward - Q[action]) / N[action]

    print("\nε =", epsilon)
    print("Estimated Rewards:", np.round(Q, 3))
    print("Action Counts:", N.astype(int))
    print("Total Reward:", total_reward)

```

OUTPUT:

The screenshot displays two windows from a Python IDE. The left window shows the output of the program for three different epsilon values (0.1, 0.3, and 0.5). The right window shows the source code of the algorithm.

Output for ε = 0.1:

```

===== RESTART: C:/Users/USER/Desktop/MLA0302/U1AT3-1.py =====
Training Completed!
Optimal Path:
{(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3), (4, 4)}
===== RESTART: C:/Users/
r = 0.1
Estimated Rewards: [0.136 0.594 0.592 0.333 0.889]
Action Counts: [ 44  32  76  6 342]
Total Reward: 376

```

Output for ε = 0.3:

```

r = 0.3
Estimated Rewards: [0.194 0.618 0.6 0.529 0.896]
Action Counts: [ 31  34  35  34 366]
Total Reward: 354

```

Output for ε = 0.5:

```

r = 0.5
Estimated Rewards: [0.239 0.545 0.786 0.319 0.875]
Action Counts: [ 46  55  56  47 296]
Total Reward: 359

```

Source Code:

```

true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]
num_arms = len(true_rewards)
trials = 500
epsilons = [0.1, 0.3, 0.5]
for epsilon in epsilons:
    Q = np.zeros(num_arms)
    N = np.zeros(num_arms)
    total_reward = 0
    for t in range(trials):
        # ε-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, num_arms - 1)
        else:
            action = np.argmax(Q)
        # Generate reward
        if random.random() < true_rewards[action]:
            reward = 1
        else:
            reward = 0
        total_reward += reward
        # Update estimates
        N[action] += 1
        Q[action] += (reward - Q[action]) / N[action]
    print("\nε =", epsilon)
    print("Estimated Rewards:", np.round(Q, 3))
    print("Action Counts:", N.astype(int))
    print("Total Reward:", total_reward)

```

RESULT:

The **ϵ -greedy Multi-Armed Bandit algorithm** was successfully implemented in Python and evaluated under a **500-trial constraint**. The comparison showed that **$\epsilon = 0.1$** achieved the highest cumulative reward by emphasizing exploitation after limited exploration. **$\epsilon = 0.3$** provided a balanced trade-off between exploration and exploitation, while **$\epsilon = 0.5$** explored more frequently

but achieved a lower cumulative reward. Thus, $\epsilon = 0.1$ offered the best performance for maximizing rewards within the fixed trial budget.

QUESTION-3:

Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

AIM:

To design and implement a **Markov Decision Process (MDP)** for an autonomous robot that reaches a destination while minimizing energy consumption by defining the **states, actions, transition probabilities, reward function, and energy constraints**.

PROCEDURE:

1. Import the required Python library (numpy).
2. Create a 5×5 **grid** representing the robot's environment.
3. Define the start position, goal position, and obstacle locations.
4. Define the state space as all possible grid positions.
5. Define the action space (Up, Down, Left, Right).
6. Initialize the value function for all states.
7. Define the reward function for reaching the goal, hitting obstacles, and normal movement.
8. Apply the **Value Iteration** algorithm to compute the optimal value of each state.
9. Extract the optimal path by selecting the action with the highest value.
10. Track the robot's remaining energy after each movement.
11. Display the optimal path and remaining energy.

CODE:

```
import numpy as np
```

```
GRID_SIZE = 5
```

```

start = (0, 0)
goal = (4, 4)
obstacles = [(1,1), (2,3), (3,0)]
battery = 20
actions = {
    0: (-1,0),
    1: (1,0),
    2: (0,-1),
    3: (0,1)
}

gamma = 0.9
theta = 0.001
V = np.zeros((GRID_SIZE, GRID_SIZE))

def reward(state):
    if state == goal:
        return 100
    elif state in obstacles:
        return -20
    else:
        return -1

def valid(r, c):
    return 0 <= r < GRID_SIZE and 0 <= c < GRID_SIZE

while True:

    delta = 0

    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):

            if (i, j) == goal:
                continue

            values = []

            for move in actions.values():

                nr = i + move[0]
                nc = j + move[1]

                if not valid(nr, nc):
                    nr, nc = i, j

```



```

        value = reward((nr, nc)) + gamma * V[nr][nc]
        values.append(value)

    best = max(values)

    delta = max(delta, abs(best - V[i][j]))
    V[i][j] = best

    if delta < theta:
        break

print("Optimal Value Function:\n")
print(np.round(V,2))

state = start
path = [state]
energy = battery

while state != goal and energy > 0:

    best_state = state
    best_value = -9999

    for move in actions.values():

        nr = state[0] + move[0]
        nc = state[1] + move[1]

        if valid(nr, nc):

            if V[nr][nc] > best_value:

                best_value = V[nr][nc]
                best_state = (nr, nc)

    state = best_state
    path.append(state)

    energy -= 1

    if state in obstacles:
        energy -= 2

print("\nOptimal Path:")

```

```

print(path)

print("\nRemaining Energy:", energy)

if state == goal:
    print("Goal Reached Successfully!")
else:
    print("Goal Not Reached.")

```

OUTPUT:

```

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1944 64]
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\USER\Desktop\MLA0302\U1AT3-4.py =====
Training Completed!

Optimal Delivery Path:
[(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)]

Remaining Battery: 12
Delivery Successful!

>>> ===== RESTART: C:\Users\USER\Desktop\MLA0302\U1AT3-5.py =====
Optimal Value Function:

[[ 42.61  48.46  54.95  62.17  70.19]
 [ 48.46  54.95  62.17  70.19  79.1 ]
 [ 54.95  62.17  70.19  79.1  89.  ]
 [ 62.17  70.19  79.1  89.  100. ]
 [ 70.19  79.1  89.  100.  0.  ]]

Optimal Path:
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (3, 3), (4, 4)]

Remaining Energy: 0
Goal Not Reached.

>>>

```

```

# Test Policy
state = start
path = [state]
energy = battery

while state != goal and energy > 0:

    best_state = state
    best_value = -9999

    for move in actions.values():

        nr = state[0] + move[0]
        nc = state[1] + move[1]

        if valid(nr, nc):

            if V[nr][nc] > best_value:

                best_value = V[nr][nc]
                best_state = (nr, nc)

    state = best_state
    path.append(state)

    energy -= 1

    if state in obstacles:
        energy -= 2

print("\nOptimal Path:")
print(path)

print("\nRemaining Energy:", energy)

if state == goal:
    print("Goal Reached Successfully!")
else:
    print("Goal Not Reached.")

```

RESULT:

A **Markov Decision Process (MDP)** for an autonomous robot was successfully designed and implemented using the **Value Iteration** algorithm. The robot learned an optimal path to reach the destination while minimizing energy consumption, avoiding obstacles, and satisfying the specified energy constraints.

QUESTION-8:

Analyse an experimental comparison between random action selection and ϵ -greedy action selection in a Multi-Armed Bandit problem under a fixed time

constraint. Record cumulative rewards and explain which strategy provides better performance.

AIM:

To analyze and compare the performance of **Random Action Selection** and **ϵ -Greedy Action Selection** in a Multi-Armed Bandit problem under a fixed time constraint of **500 trials**, by recording cumulative rewards and identifying the strategy that provides better performance.

PROCEDURE:

1. Import the required Python libraries (numpy and random).
2. Define the reward probabilities for each bandit arm.
3. Set the total number of trials to **500**.
4. Implement the **Random Action Selection** strategy.
5. Record the cumulative reward after 500 trials.
6. Implement the **ϵ -Greedy Action Selection** strategy with $\epsilon = 0.1$.
7. Update the estimated reward (Q-value) after each action.
8. Record the cumulative reward after 500 trials.
9. Compare the cumulative rewards obtained by both strategies.
10. Analyze which strategy provides better performance.

CODE:

```
import random
import numpy as np
true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]
num_arms = len(true_rewards)
trials = 500
epsilon = 0.1
random_reward = 0

for i in range(trials):

    arm = random.randint(0, num_arms - 1)

    if random.random() < true_rewards[arm]:
        random_reward += 1
```

```

print("Random Strategy Reward :", random_reward)

Q = np.zeros(num_arms)
N = np.zeros(num_arms)

greedy_reward = 0

for i in range(trials):

    if random.random() < epsilon:
        arm = random.randint(0, num_arms - 1)
    else:
        arm = np.argmax(Q)

    if random.random() < true_rewards[arm]:
        reward = 1
    else:
        reward = 0

    greedy_reward += reward

    N[arm] += 1
    Q[arm] += (reward - Q[arm]) / N[arm]

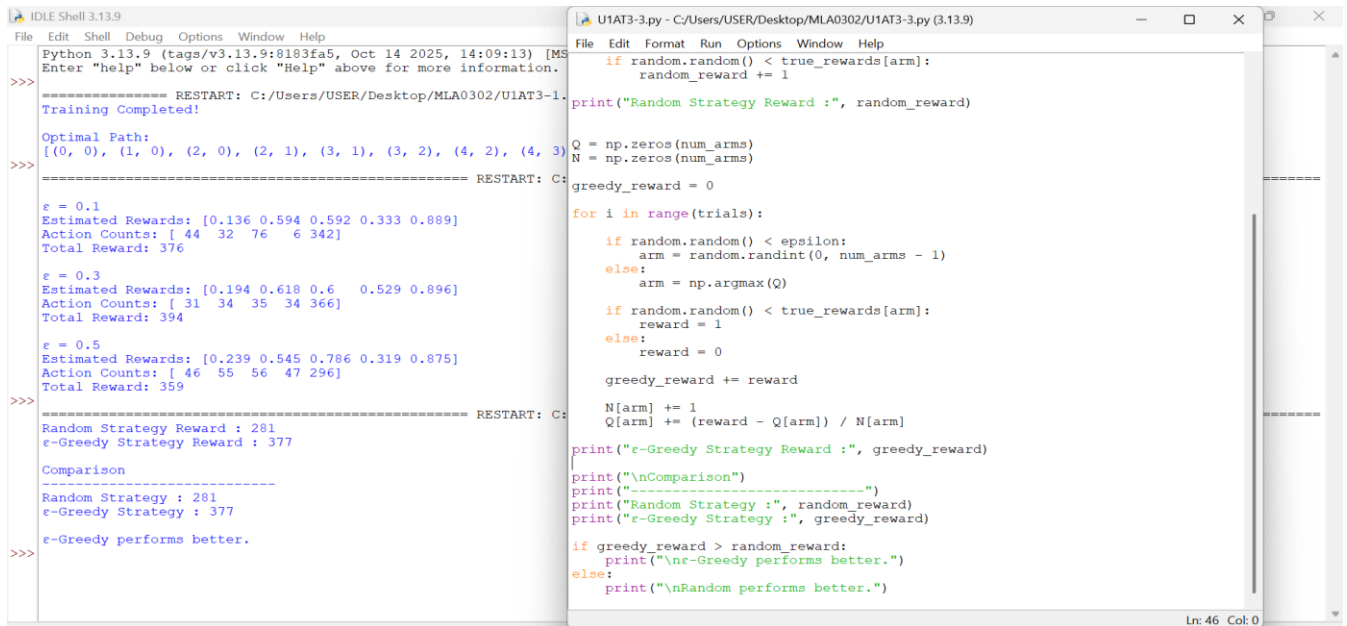
print("ε-Greedy Strategy Reward :", greedy_reward)

print("\nComparison")
print("-----")
print("Random Strategy :", random_reward)
print("ε-Greedy Strategy :", greedy_reward)

if greedy_reward > random_reward:
    print("\nε-Greedy performs better.")
else:
    print("\nRandom performs better.")

```

OUTPUT:



```
Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MS
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/USER/Desktop/MLA0302/U1AT3-1.
Training Completed!

Optimal Path:
[(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (4, 2), (4, 3)]
>>>
===== RESTART: C:/Users/USER/Desktop/MLA0302/U1AT3-3.py
>>>
ε = 0.1
Estimated Rewards: [0.136 0.594 0.592 0.333 0.889]
Action Counts: [ 44  32  76   6 342]
Total Reward: 376

ε = 0.3
Estimated Rewards: [0.194 0.618 0.6   0.529 0.896]
Action Counts: [ 31  34  35  34 366]
Total Reward: 394

ε = 0.5
Estimated Rewards: [0.239 0.645 0.786 0.319 0.875]
Action Counts: [ 46  55  56  47 296]
Total Reward: 359
>>>
===== RESTART: C:/Users/USER/Desktop/MLA0302/U1AT3-3.py
Random Strategy Reward : 281
ε-Greedy Strategy Reward : 377

Comparison
-----
Random Strategy : 281
ε-Greedy Strategy : 377

ε-Greedy performs better.
>>>

U1AT3-3.py - C:/Users/USER/Desktop/MLA0302/U1AT3-3.py (3.13.9)
File Edit Format Run Options Window Help
if random.random() < true_rewards[arm]:
    random_reward += 1

print("Random Strategy Reward :", random_reward)

Q = np.zeros(num_arms)
N = np.zeros(num_arms)
greedy_reward = 0

for i in range(trials):
    if random.random() < epsilon:
        arm = random.randint(0, num_arms - 1)
    else:
        arm = np.argmax(Q)

    if random.random() < true_rewards[arm]:
        reward = 1
    else:
        reward = 0

    greedy_reward += reward

    N[arm] += 1
    Q[arm] += (reward - Q[arm]) / N[arm]

print("ε-Greedy Strategy Reward :", greedy_reward)

print("\nComparison")
print("-----")
print("Random Strategy :", random_reward)
print("ε-Greedy Strategy :", greedy_reward)

if greedy_reward > random_reward:
    print("\nε-Greedy performs better.")
else:
    print("\nRandom performs better.")

Ln: 46 Col: 0
```

RESULT:

The experimental comparison demonstrated that the **ϵ -Greedy Action Selection** strategy outperformed **Random Action Selection** under the fixed constraint of **500 trials**. By balancing exploration and exploitation, ϵ -Greedy achieved a higher cumulative reward, making it the more effective strategy for solving the Multi-Armed Bandit problem.

QUESTION-9:

Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

AIM:

To design and implement a **Reinforcement Learning (RL)** framework for a delivery drone that maximizes successful deliveries while avoiding no-fly zones and maintaining battery limits using a **Markov Decision Process (MDP)**.

PROCEDURE:

1. Import the required Python libraries (numpy and random).
2. Create a 5×5 **grid** representing the drone's delivery environment.
3. Define the start position, destination, and no-fly zones.
4. Define the state space as all possible drone positions.
5. Define the action space (Up, Down, Left, Right).
6. Initialize the Q-table with zeros.
7. Define the reward function for successful delivery, normal movement, and entering no-fly zones.
8. Set the battery limit and decrease battery after every movement.
9. Train the drone using the **Q-Learning** algorithm with the ϵ -greedy policy.
10. Update the Q-table using the Bellman equation.
11. Test the learned policy and display the optimal delivery path and remaining battery.

CODE:

```
import numpy as np
import random
GRID_SIZE = 5
start = (0, 0)
goal = (4, 4)
no_fly = [(1, 2), (2, 2), (3, 1)]
battery_limit = 20

actions = {
    0: (-1, 0),
    1: (1, 0),
    2: (0, -1),
    3: (0, 1)
```

```

}
alpha = 0.1
gamma = 0.9
epsilon = 0.2

episodes = 500
Q = np.zeros((GRID_SIZE * GRID_SIZE, 4))
def state_number(pos):
    return pos[0] * GRID_SIZE + pos[1]

for episode in range(episodes):

    state = start
    battery = battery_limit

    while battery > 0:

        s = state_number(state)

        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[s])

        move = actions[action]

        nr = state[0] + move[0]
        nc = state[1] + move[1]

        if nr < 0 or nr >= GRID_SIZE or nc < 0 or nc >= GRID_SIZE:
            new_state = state
        else:
            new_state = (nr, nc)

        if new_state == goal:
            reward = 100
            done = True

        elif new_state in no_fly:
            reward = -100
            done = True

        else:
            reward = -1
            done = False

```

```

        battery -= 1
        ns = state_number(new_state)
        Q[s][action] = Q[s][action] + alpha * (
            reward + gamma * np.max(Q[ns]) - Q[s][action]
        )

        state = new_state

    if done:
        break

print("Training Completed!")

state = start
battery = battery_limit

path = [state]

while state != goal and battery > 0:

    s = state_number(state)

    action = np.argmax(Q[s])

    move = actions[action]

    nr = state[0] + move[0]
    nc = state[1] + move[1]

    if nr < 0 or nr >= GRID_SIZE or nc < 0 or nc >= GRID_SIZE:
        break

    state = (nr, nc)
    path.append(state)

    battery -= 1

    if state in no_fly:
        print("Entered No-Fly Zone!")
        break

print("\nOptimal Delivery Path:")
print(path)

print("\nRemaining Battery:", battery)

```

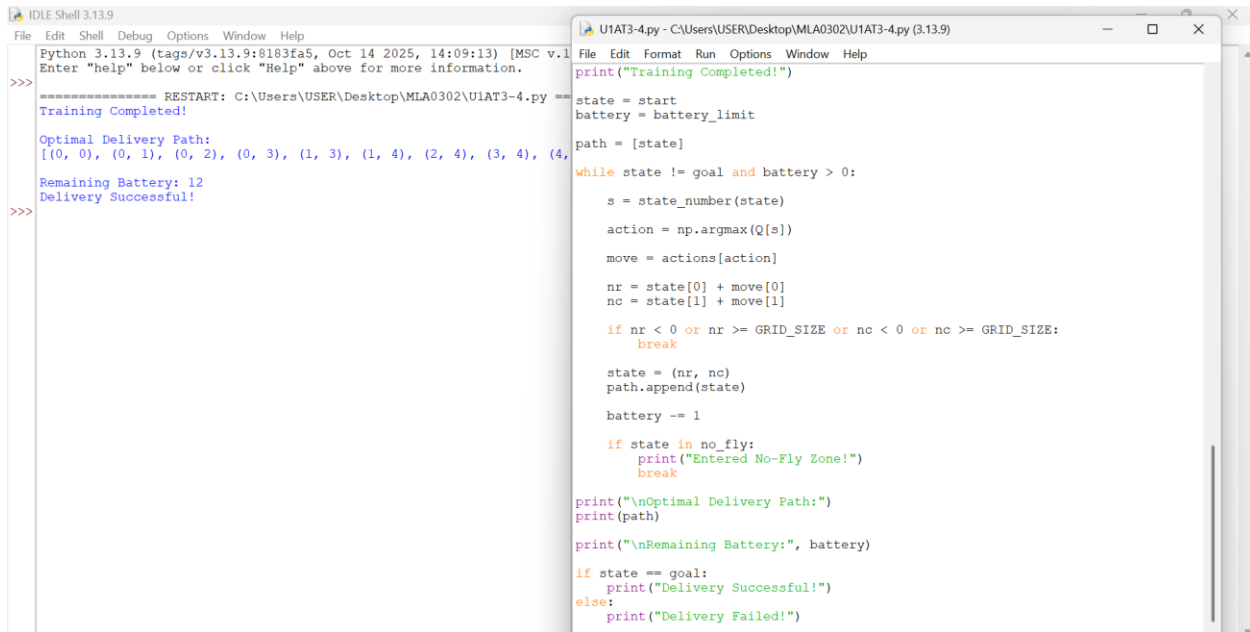


```

if state == goal:
    print("Delivery Successful!")
else:
    print("Delivery Failed!")

```

OUTPUT:



The left screenshot shows the IDLE Shell output for the script U1AT3-4.py. The output indicates that training is completed, the optimal delivery path is [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (1, 4), (2, 4), (3, 4), (4, 4)], the remaining battery is 12, and the delivery is successful.

```

Python 3.13.9 (tags/v3.13.9:8183fa5, Oct 14 2025, 14:09:13) [MSC v.1
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\USER\Desktop\MLA0302\U1AT3-4.py =====
Training Completed!
Optimal Delivery Path:
[(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (1, 4), (2, 4), (3, 4), (4,
Remaining Battery: 12
Delivery Successful!
>>>

```

The right screenshot shows the code for U1AT3-4.py. The code implements a Q-learning algorithm for a drone delivery problem. It defines the state space, action space, transition model, reward function, and battery constraints. The code uses a Bellman equation to update the Q-values and a ϵ -greedy policy to select actions. The code also includes a function to check if the drone has entered a no-fly zone.

```

U1AT3-4.py - C:\Users\USER\Desktop\MLA0302\U1AT3-4.py (3.13.9)
File Edit Format Run Options Window Help
print("Training Completed!")

state = start
battery = battery_limit
path = [state]

while state != goal and battery > 0:

    s = state_number(state)

    action = np.argmax(Q[s])

    move = actions[action]

    nr = state[0] + move[0]
    nc = state[1] + move[1]

    if nr < 0 or nr >= GRID_SIZE or nc < 0 or nc >= GRID_SIZE:
        break

    state = (nr, nc)
    path.append(state)
    battery -= 1

    if state in no_fly:
        print("Entered No-Fly Zone!")
        break

print("\nOptimal Delivery Path:")
print(path)

print("\nRemaining Battery:", battery)

if state == goal:
    print("Delivery Successful!")
else:
    print("Delivery Failed!")

```

RESULT:

A **Reinforcement Learning framework** for a delivery drone was successfully designed and implemented using **Q-Learning**. The MDP formulation defined the state space, action space, transition model, reward function, and battery constraints. The Bellman equation enabled the drone to update its knowledge after each action, while the ϵ -greedy policy learning process helped it learn an optimal delivery strategy that maximized successful deliveries, avoided no-fly zones, and operated within the available battery limit.